

OpenShift UPI Install on vSphere — Airgapped, Production Runbook

Mirror Registry + HAProxy LB + Content Library RHCOS deployment

0. vSphere terms you need (coming from bare-metal)

Term	Meaning
vCenter Server (vCSA)	Management plane for all ESXi hosts — like a control panel for the virtual datacenter.
ESXi host	The hypervisor server running the VMs (your “bare-metal box” equivalent).
Datacenter	Logical container object in vCenter, not a physical building.
Cluster (vSphere)	Group of pooled ESXi hosts — not the OpenShift cluster. Don’t conflate the two.
Resource Pool	CPU/RAM carve-out inside a vSphere cluster.
Datastore	Shared storage (VMFS/NFS/vSAN) backing VM disks.
Portgroup	Virtual switch network a VM’s NIC attaches to ($\approx$ VLAN/subnet).
Content Library	A storage repository in vCenter for OVF/OVA templates and ISOs — upload once, deploy many times.
govc	CLI for vCenter (optional here since you’re using the GUI).

1. Architecture

- 1x **Bootstrap** VM (temporary)
- 3x **Control plane (master)** VMs

- 2+ **Worker** VMs
- 1x **HAProxy** VM (LB for API + Ingress, outside the cluster)
- 1x **Mirror registry** VM (OCP release + operator images)
- DNS with the required records (existing infra DNS or a small VM)
- DHCP scope on the node portgroup handing out the correct DNS server

All node VMs are cloned from a single RHCOS image stored in a vCenter **Content Library**.

2. Files to download (on an internet-connected machine, transfer across the airgap)

File	Source
<code>openshift-install-linux.tar.gz</code>	console.redhat.com/openshift/downloads
<code>openshift-client-linux.tar.gz (oc)</code>	console.redhat.com/openshift/downloads
Pull secret	console.redhat.com/openshift/install/pull-secret
<code>rhcos-<version>-x86_64-vmware.x86_64.ova</code>	<a href="https://mirror.openshift.com/pub/openshift-v4/dependencies/rhcos/<version>/">mirror.openshift.com/pub/openshift-v4/dependencies/rhcos/ <version> /
<code>mirror-registry.tar.gz</code>	console.redhat.com → Downloads → “mirror registry for Red Hat OpenShift”
<code>oc-mirror.tar.gz (oc-mirror plugin)</code>	console.redhat.com/openshift/downloads

Pin the RHCOS OVA version to match your target OCP minor version exactly — mismatches cause bootstrap to hang with cryptic ignition errors.

3. Mirror Registry VM

Spec: 4 vCPU / 16GB RAM / 200GB+ disk, RHEL 8/9, static IP + DNS entry.

```
tar xvf mirror-registry.tar.gz
sudo ./mirror-registry install \
  --quayHostname mirror-registry.example.com \
  --quayRoot /opt/quay-install
```

Note the printed admin credentials and the auto-generated `rootCA.pem`.

On your bastion host (used for all `oc / oc-mirror` commands):

```
cp rootCA.pem /etc/pki/ca-trust/source/anchors/mirror-registry-ca.pem
update-ca-trust extract
```

4. Mirror OpenShift images with oc-mirror

`imageset-config.yaml` (production example — pin exact version, not just channel):

```
apiVersion: mirror.openshift.io/v1alpha2
kind: ImageSetConfiguration
storageConfig:
  local:
    path: ./oc-mirror-workspace
mirror:
  platform:
    channels:
      - name: stable-4.16
        type: ocp
        minVersion: 4.16.18
        maxVersion: 4.16.18
  operators:
    - catalog: registry.redhat.io/redhat/redhat-operator-index:v4.16
      packages:
        - name: local-storage-operator
        - name: cluster-logging
        - name: elasticsearch-operator
        - name: odf-operator # skip if not using ODF
  additionalImages:
    - name: registry.redhat.io/ubi8/ubi:latest
```

On the connected machine:

```
oc mirror --config imageset-config.yaml file://mirror-export
```

Transfer `mirror-export/` across the airgap to the bastion, then push into your registry:

```
oc mirror --from=file://mirror-export \
  docker://mirror-registry.example.com:8443
```

This generates in the working dir:

- `oc-mirror-workspace/results-<timestamp>/imageContentSourcePolicy.yaml`
- `oc-mirror-workspace/results-<timestamp>/catalogSource.yaml`
- an updated pull secret snippet to merge with your Red Hat pull secret

Save all three — needed in Step 8.

5. HAProxy LB VM

Spec: 2 vCPU / 4GB RAM, RHEL/CentOS minimal, static IP + DNS entry for `api / *.apps .`

```
dnf install -y haproxy
```

`/etc/haproxy/haproxy.cfg` (production — remove bootstrap lines after Step 11):

```
global
    log          127.0.0.1 local2
    maxconn      4000

defaults
    mode          tcp
    log           global
    option        tcplog
    option        dontlognull
    retries       3
    timeout http-request 10s
    timeout queue 1m
    timeout connect 10s
    timeout client 1m
    timeout server 1m
    timeout http-keep-alive 10s
    timeout check 10s
    maxconn       3000

frontend api
    bind *:6443
```

```

    default_backend api-backend

backend api-backend
    balance source
    server bootstrap <bootstrap-ip>:6443 check
    server master0 <master0-ip>:6443 check
    server master1 <master1-ip>:6443 check
    server master2 <master2-ip>:6443 check

frontend machine-config
    bind *:22623
    default_backend machine-config-backend

backend machine-config-backend
    balance source
    server bootstrap <bootstrap-ip>:22623 check
    server master0 <master0-ip>:22623 check
    server master1 <master1-ip>:22623 check
    server master2 <master2-ip>:22623 check

frontend http
    bind *:80
    default_backend http-backend

backend http-backend
    balance source
    server worker0 <worker0-ip>:80 check
    server worker1 <worker1-ip>:80 check

frontend https
    bind *:443
    default_backend https-backend

backend https-backend
    balance source
    server worker0 <worker0-ip>:443 check
    server worker1 <worker1-ip>:443 check

setsebool -P haproxy_connect_any 1
systemctl enable --now haproxy

```

6. DNS records required

Record	Type	Target
api.<cluster>.<domain>	A	HAProxy IP
api-int.<cluster>.<domain>	A	HAProxy IP
*.apps.<cluster>.<domain>	A	HAProxy IP
bootstrap.<cluster>.<domain>	A	bootstrap IP
master0/1/2.<cluster>.<domain>	A	master IPs
worker0..N.<cluster>.<domain>	A	worker IPs
etcd-0/1/2.<cluster>.<domain>	A	master0/1/2 IPs
_etcd-server-ssl._tcp.<cluster>.<domain>	SRV	etcd-0/1/2 : port 2380, priority/weight 0 10

DNS is resolved via DHCP, not set on the VM — the DHCP server for the node portgroup must hand out this DNS server's IP in its options. If you're on static IPs instead, this is set via ignition network config, not covered in this doc — flag it if that's your case.

7. NTP — MachineConfig manifests (airgapped nodes can't reach public NTP pools)

99-master-chrony.yaml :

```

apiVersion: machineconfiguration.openshift.io/v1
kind: MachineConfig
metadata:
  labels:
    machineconfiguration.openshift.io/role: master
  name: 99-master-chrony
spec:
  config:
    ignition:
      version: 3.2.0
    storage:
      files:
        - path: /etc/chrony.conf
          mode: 0644
          overwrite: true

```

```
contents:
  source: data:text/plain;charset=utf-8;base64,c2VydmVyIG50cC5pbmRlcm5h
```

Duplicate as 99-worker-chrony.yaml with role: worker .

To regenerate the base64 for your own NTP server:

```
cat <<EOF | base64 -w0
server ntp.internal.example.com iburst
driftfile /var/lib/chrony/drift
makestep 1.0 3
rtcsync
EOF
```

Paste the output into the source: data:text/plain;charset=utf-8;base64, field.

8. install-config.yaml (production)

```
apiVersion: v1
baseDomain: example.com
compute:
- name: worker
  replicas: 0          # UPI = 0, workers created manually
controlPlane:
  name: master
  replicas: 3
metadata:
  name: prod-ocp
platform:
  vsphere:
    apiVIPs: []
    ingressVIPs: []
    vcenters:
    - server: vcenter.example.com
      user: svc-ocp-installer@vsphere.local
      password: <password>
      datacenter: <datacenter>
  failureDomains:
  - name: generated-failure-domain
    region: generated-region
    zone: generated-zone
    server: vcenter.example.com
    topology:
```

```

datacenter: <datacenter>
computeCluster: /<datacenter>/host/<vsphere-cluster>
resourcePool: /<datacenter>/host/<vsphere-cluster>/Resources
networks:
- <portgroup-name>
datastore: /<datacenter>/datastore/<datastore>
folder: /<datacenter>/vm/<folder>
imageContentSources:
- mirrors:
  - mirror-registry.example.com:8443/openshift/release
  source: quay.io/openshift-release-dev/ocp-release
- mirrors:
  - mirror-registry.example.com:8443/openshift/release
  source: quay.io/openshift-release-dev/ocp-v4.0-art-dev
additionalTrustBundle: |
-----BEGIN CERTIFICATE-----
<paste mirror registry rootCA.pem content here>
-----END CERTIFICATE-----
pullSecret: '<merged pull secret from Step 4>'
sshKey: '<bastion public ssh key>'

```

Use a **dedicated service account** (`svc-ocp-installer`) with the minimum vSphere permissions (not full Administrator) for production — Red Hat publishes the exact required privilege set; ask if you want that role list generated too.

9. Generate manifests + ignition configs

```

mkdir ocp-install && cp install-config.yaml ocp-install/
openshift-install create manifests --dir=ocp-install

sed -i 's/mastersSchedulable: true/mastersSchedulable: false/' \
  ocp-install/manifests/cluster-scheduler-02-config.yml

cp imageContentSourcePolicy.yaml ocp-install/manifests/
cp 99-master-chrony.yaml ocp-install/manifests/
cp 99-worker-chrony.yaml ocp-install/manifests/

openshift-install create ignition-configs --dir=ocp-install

```

Serve the resulting `bootstrap.ign`, `master.ign`, `worker.ign` over HTTP from the bastion (needed for guestinfo-based ignition pull on first boot):

```
cd ocp-install && python3 -m http.server 8080
```

10. RHCOS via Content Library, then clone VMs (GUI)

One-time:

1. **Menu → Content Libraries → Create** → name it, local storage backing.
2. Open the library → **Actions → Import Item** → upload `rhcos-<version>-x86_64-vmware.x86_64.ova`.

Per node (bootstrap, master0-2, worker0-N):

1. Right-click the OVA item in the library → **New VM from This Template**.
2. Walk the wizard: name, folder, compute resource, storage, network (select your portgroup). Skip any guest OS customization spec — ignition handles config, not vSphere customization.
3. Finish with the VM **powered off**.
4. **Edit Settings → VM Options → Advanced → Edit Configuration → Add Parameter**, add all three:

Key	Value
<code>guestinfo.ignition.config.data</code>	base64 of the matching <code>.ign</code> file (<code>base64 -w0 bootstrap.ign</code> etc.)
<code>guestinfo.ignition.config.data.encoding</code>	base64
<code>disk.EnableUUID</code>	TRUE (required later by the vSphere CSI driver — easy to miss)

5. Edit Settings → CPU/Memory:

- Bootstrap: 4 vCPU / 16GB
 - Masters: 4 vCPU / 16GB, 120GB disk min
 - Workers: 2 vCPU / 8GB, 120GB disk min
6. Power on.
-

11. Bootstrap monitoring

From the bastion:

```
export KUBECONFIG=ocp-install/auth/kubeconfig
openshift-install wait-for bootstrap-complete --dir=ocp-install --log-level=info
```

Once it reports "It is now safe to remove the bootstrap resources":

- Power off and delete the bootstrap VM
- Remove all `bootstrap` lines from `haproxy.cfg`, then:

```
systemctl reload haproxy
```

12. Approve CSRs and finish install

```
oc get csr -o name | xargs oc adm certificate approve
openshift-install wait-for install-complete --dir=ocp-install
```

Verify:

```
oc get nodes
oc get co
oc get clusterversion
```

All ClusterOperators should show `AVAILABLE=True`, `PROGRESSING=False`, `DEGRADED=False`.

13. Post-install — vSphere CSI storage (production requirement)

Since you set `disk.EnableUUID=TRUE` on every node, the in-tree/CSI vSphere storage driver ships enabled by default in OCP 4.16 — just verify:

```
oc get storageclass
oc get pods -n openshift-cluster-csi-drivers
```

If it's not auto-configuring, it usually means a vCenter permission or `disk.EnableUUID` was

missed on one node — check that node first.

Order of operations (checklist)

- Mirror registry VM up, OCP images mirrored in (Steps 3-4)
- HAProxy VM up, bootstrap+master backends only (Step 5)
- DNS records created; DHCP scope handing out correct DNS (Step 6)
- chrony MachineConfigs authored (Step 7)
- install-config.yaml finalized with mirror + trust bundle (Step 8)
- manifests + ignition generated, ICSP + chrony configs injected (Step 9)
- RHCOS uploaded to Content Library once (Step 10)
- Bootstrap + 3 masters + workers cloned, ignition injected, powered on (Step 10)
- wait-for bootstrap-complete → remove bootstrap VM + haproxy lines (Step 11)
- Approve CSRs → wait-for install-complete (Step 12)
- Verify nodes, cluster operators, storage (Step 13)